

## Projet Optimisation Combinatoire

Master 1 MIAGE (Mixte)

# Optimisation des tournées en entrepôt : Hybridation du problème du Voyageur de Commerce (TSP) et du Clustering

Rapport réalisé du Mars 2026 au Avril 2026

*présenté par*

Kenza MENAD, Lyna BAOUCHE, Dyhia SELLAH

# Table des matières

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>3</b>  |
| <b>2</b> | <b>Intégration du Machine Learning</b>                  | <b>4</b>  |
| 2.1      | Les algorithmes de clustering évalués . . . . .         | 4         |
| 2.2      | Sélection automatique du nombre de clusters k . . . . . | 4         |
| 2.3      | Métriques d'évaluation des clusterings . . . . .        | 5         |
| 2.4      | Résultats et comparaison . . . . .                      | 5         |
| 2.5      | Visualisations . . . . .                                | 6         |
| 2.6      | Choix de ML - K-Means . . . . .                         | 7         |
| 2.7      | Intégration avec la métaheuristique . . . . .           | 8         |
| <b>3</b> | <b>Métaheuristiques et Optimisation</b>                 | <b>9</b>  |
| 3.1      | Implémentation des Algorithmes . . . . .                | 9         |
| 3.1.1    | Algorithme des Colonies de Fourmis (ACO) . . . . .      | 9         |
| 3.1.2    | Algorithme Génétique (AG) . . . . .                     | 9         |
| 3.2      | Résultats et Comparaison des Approches . . . . .        | 9         |
| 3.2.1    | Tableau de synthèse . . . . .                           | 10        |
| 3.2.2    | Analyse des performances . . . . .                      | 10        |
| 3.2.3    | Visualisation des performances . . . . .                | 10        |
| 3.3      | Interprétation . . . . .                                | 11        |
| <b>4</b> | <b>Conclusion</b>                                       | <b>12</b> |
| <b>5</b> | <b>Ressources et code source</b>                        | <b>13</b> |

# Chapitre 1

## Introduction

La préparation de commandes, ou *Order Picking*, est l'une des opérations les plus coûteuses dans la gestion d'un entrepôt. Elle représente en moyenne 55% des coûts opérationnels d'un entrepôt, et consiste à collecter des articles dispersés dans différentes zones pour constituer des commandes clients. Optimiser les déplacements du préparateur permet de réduire significativement ces coûts et d'améliorer la productivité.

Ce problème peut être modélisé comme un **Problème du Voyageur de Commerce** (TSP — *Travelling Salesman Problem*) : le préparateur doit visiter un ensemble de points (les articles à collecter) en parcourant la distance la plus courte possible, en partant et en revenant au dépôt. Cependant, lorsque le nombre d'articles est élevé (environ 119 articles dans notre cas), l'espace de recherche explose il y a  $119!$  tournées possibles ce qui rend une résolution exacte impossible en temps raisonnable.

Pour faire face à cette complexité, nous proposons une **approche hybride** combinant deux familles de méthodes complémentaires :

- Le **Machine Learning** (clustering K-Means) pour décomposer intelligemment le problème en sous-problèmes plus petits, en regroupant les articles géographiquement proches en batches cohérents.
- Les **Métaheuristiques** (Algorithme des Colonies de Fourmis et Algorithme Génétique) pour optimiser les tournées à l'intérieur de chaque batch, et comparer leurs performances.

L'idée centrale est d'utiliser le clustering **en amont** de la métaheuristique : plutôt que de résoudre un TSP global de 119 points, on résout 4 sous-TSP d'environ 30 articles chacun, ce qui réduit drastiquement la complexité du problème.

# Chapitre 2

## Intégration du Machine Learning

Notre problème consiste à minimiser la distance parcourue par un préparateur de commandes dans un entrepôt de  $200 \times 200$  unités, avec 119 articles à collecter. Si on soumet directement ces points à une métaheuristique (ACO ou algorithme génétique), on obtient un TSP de grande taille : l'espace de recherche explose ( $119!$  permutations possibles) et la convergence est très lente.

Notre solution : utiliser le Machine Learning en amont pour regrouper les articles géographiquement proches en batches. Chaque batch devient alors un sous-TSP de petite taille, que la métaheuristique résout rapidement.

### 2.1 Les algorithmes de clustering évalués

Le clustering est une méthode d'apprentissage non supervisé : elle regroupe des données en groupes homogènes (clusters) sans étiquette préalable. Ici, chaque point est la coordonnée  $(x, y)$  d'un article. Nous avons comparé quatre algorithmes de familles différentes :

- **K-Means** : algorithme de partitionnement itératif basé sur la minimisation de la variance intra-cluster (WCSS). Il assigne chaque point au centroïde le plus proche, puis recalcule les centroïdes, et répète jusqu'à convergence.
- **Agglomerative Clustering** : algorithme hiérarchique ascendant. Il commence avec chaque point dans son propre cluster, puis fusionne progressivement les clusters les plus proches selon un critère de liaison (ici : Ward).
- **GMM — Gaussian Mixture Model (EM)** : modèle probabiliste qui suppose que les données sont générées par un mélange de distributions gaussiennes. L'algorithme Expectation-Maximization (EM) estime les paramètres de chaque gaussienne.
- **DBSCAN — Density-Based Spatial Clustering** : algorithme basé sur la densité. Il regroupe les points denses en clusters et marque comme bruit les points isolés. Il ne nécessite pas de spécifier  $k$  à l'avance.

### 2.2 Sélection automatique du nombre de clusters $k$

K-Means, Agglomerative et GMM nécessitent de fixer  $k$  avant l'exécution. Pour choisir  $k$  automatiquement, nous utilisons le **Silhouette Score**, calculé pour  $k$  allant de 2 à 8, et nous retenons la valeur qui le maximise.

Le Silhouette Score mesure à quel point chaque point est bien placé dans son cluster :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (2.1)$$

où  $a(i)$  est la distance moyenne au sein du cluster (cohésion) et  $b(i)$  est la distance au cluster voisin le plus proche (séparation). Un score proche de 1 indique des clusters bien séparés. Sur nos données (119 articles, `seed=42`), le score est maximal pour  $k = 4$  avec un Silhouette Score de 0,404.

## 2.3 Métriques d'évaluation des clusterings

Pour comparer les algorithmes de façon rigoureuse, nous utilisons trois métriques complémentaires :

| Métrique               | Sens    | Ce que ça mesure dans notre contexte                                                                         |
|------------------------|---------|--------------------------------------------------------------------------------------------------------------|
| Silhouette Score       | ↑ mieux | Séparation entre clusters. Score élevé = articles d'un même batch proches → tournées locales courtes.        |
| Davies-Bouldin         | ↓ mieux | Compacité des clusters par rapport à leur distance mutuelle. Index faible = clusters denses et bien séparés. |
| Calinski-Harabasz      | ↑ mieux | Densité intra-cluster vs dispersion inter-clusters. Score élevé = clusters bien distincts.                   |
| Temps d'exécution (ms) | ↓ mieux | Durée du pré-traitement ML. Doit rester négligeable devant la métaheuristique.                               |

TABLE 2.1 – Métriques d'évaluation des algorithmes de clustering

## 2.4 Résultats et comparaison

Les expériences ont été menées sur un entrepôt de  $200 \times 200$  unités, 119 articles, `seed=42`,  $k=4$ , sous Python 3.13 / scikit-learn. Voici les résultats obtenus :

| Algorithme    | k | Silhouette ↑ | Davies-Bouldin ↓ | Calinski-Harabasz ↑ | Bruit | Temps (ms) |
|---------------|---|--------------|------------------|---------------------|-------|------------|
| K-Means       | 4 | 0,404        | 0,800            | 115,33              | 0 %   | 37,37      |
| Agglomerative | 4 | 0,361        | 0,810            | 96,40               | 0 %   | 39,25      |
| GMM (EM)      | 4 | 0,390        | 0,831            | 109,21              | 0 %   | 30,05      |
| DBSCAN        | 1 | N/A          | N/A              | N/A                 | 0,8 % | 8,83       |

TABLE 2.2 – Comparaison des quatre algorithmes de clustering (119 articles,  $k=4$ , `seed=42`).

- **K-Means** obtient le meilleur Silhouette Score (0,404) et le meilleur Calinski-Harabasz (115,33), confirmant que ses clusters sont à la fois compacts et bien séparés géographiquement. Son temps d'exécution de 37,4 ms est raisonnable et son partitionnement est déterministe (grâce à la graine aléatoire fixée). C'est l'algorithme retenu pour notre pipeline.
- **Agglomerative Clustering** présente le meilleur Davies-Bouldin (0,810), ce qui signifie que ses clusters sont plus compacts en termes de dispersion interne. Cependant, son temps d'exécution est légèrement plus élevé que K-Means (39,25 ms).

ms) pour un gain de Silhouette négligeable ( $-0,043$ ). Dans notre contexte où le pré-traitement doit être rapide, ce rapport coût/bénéfice est défavorable.

- **GMM (Gaussian Mixture Model)** donne les scores les plus faibles sur toutes les métriques de qualité (Silhouette 0,390, Davies-Bouldin 0,831, Calinski-Harabasz 109,21). Cela s'explique par le fait que le modèle gaussien n'est pas adapté à une distribution spatiale uniforme : les articles sont répartis aléatoirement dans l'entrepôt et ne suivent pas naturellement des distributions elliptiques gaussiennes.
- **DBSCAN a échoué dans cette configuration** : il n'a trouvé qu'un seul cluster ( $k=1$ ), regroupant tous les articles en un seul batch, ce qui rend son utilisation impossible pour notre pipeline. Ce résultat illustre une limitation connue de DBSCAN : sa sensibilité au paramètre eps. Avec une distribution uniforme des articles, la densité est homogène sur tout l'entrepôt, rendant difficile la détection de zones denses distinctes.

## 2.5 Visualisations

La figure 1 montre côte à côte les résultats des quatre algorithmes sur les mêmes données. Pour K-Means et Agglomerative, on observe 4 zones géographiquement cohérentes. Pour GMM, les frontières entre clusters sont moins nettes, ce qui se reflète dans le Silhouette Score plus bas. DBSCAN regroupe tout en un seul cluster (orange), ce qui confirme l'échec de l'algorithme.

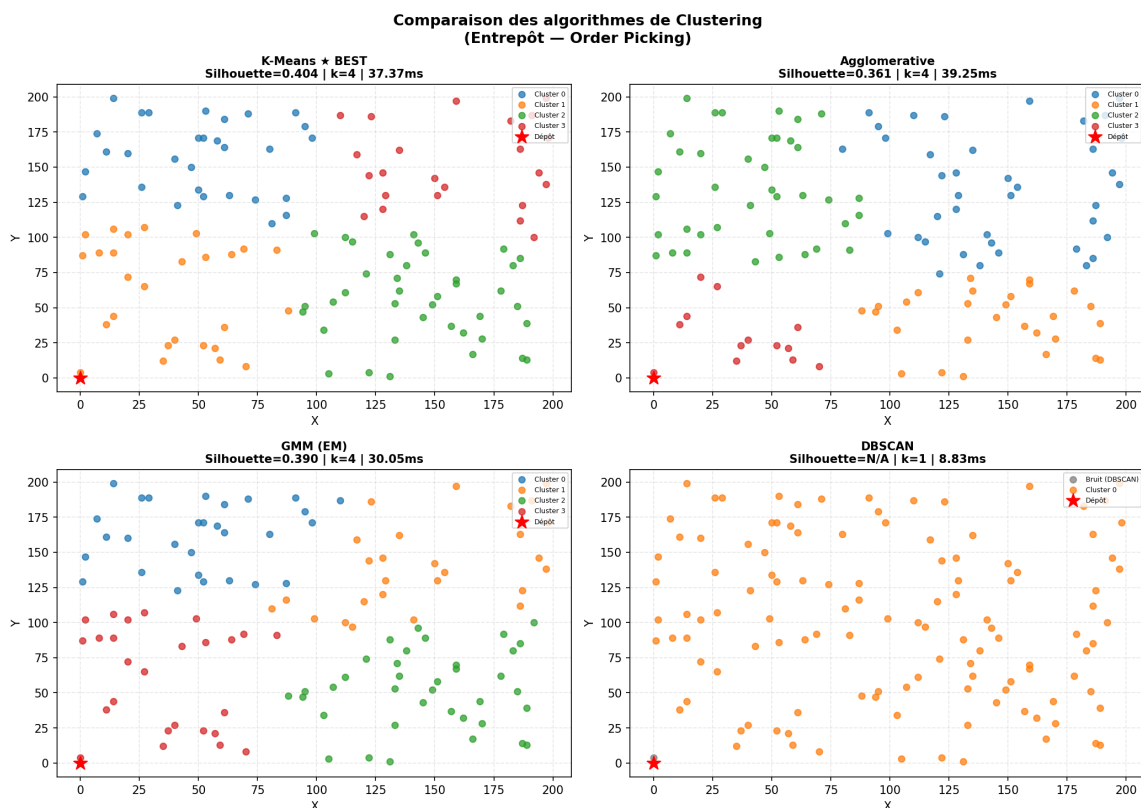


FIGURE 2.1 – Comparaison des quatre clusterings sur 119 articles (entrepôt 200×200).

La figure 2 montre que K-Means domine sur les métriques de qualité (Silhouette et Calinski-Harabasz), tout en maintenant un temps d'exécution modéré. DBSCAN

apparaît comme le plus rapide (8,83 ms), mais ses métriques de qualité sont toutes à N/A, ce qui le disqualifie.

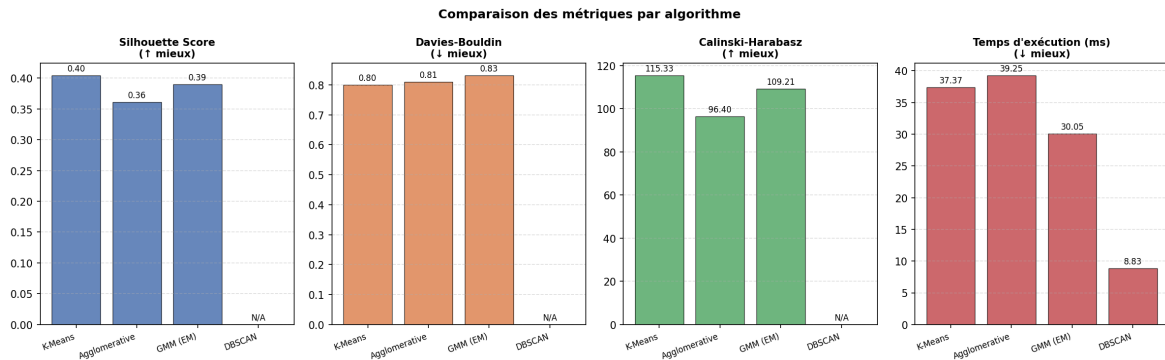


FIGURE 2.2 – Métriques comparatives par algorithme (Silhouette ↑, Davies-Bouldin ↓, Calinski-Harabasz ↑, Temps ↓).

## 2.6 Choix de ML - K-Means

Au terme de la comparaison, K-Means est retenu pour cinq raisons :

- Meilleure qualité de clustering (Silhouette 0,404 et Calinski-Harabasz 115,33) : les batches sont géographiquement cohérents.
- Résultats reproductibles grâce à la graine aléatoire fixée (seed=42), comme exigé dans le cahier des charges.
- **Intégration directe** : K-Means fournit nativement des labels et des centroïdes, exactement ce dont la métaheuristique a besoin.
- Pré-traitement rapide (37,4 ms), négligeable devant le temps de l'ACO/AG.
- **Cohérence avec la distance de Manhattan** : K-Means minimise la distance euclidienne, très proche de la distance de Manhattan dans un entrepôt à grille orthogonale.

La figure 3 présente en détail le clustering K-Means retenu. À gauche, la carte de l'entrepôt montre les 4 clusters avec leurs centroïdes et le nombre d'articles par cluster entre parenthèses. À droite, le diagramme en barres montre la distribution des tailles de clusters, avec la moyenne (environ 30 articles par batch) matérialisée par la ligne pointillée rouge. On observe que les clusters sont globalement équilibrés (de 23 à 39 articles), ce qui est favorable pour la métaheuristique : des batches de taille similaire garantissent des sous-TSP comparables en complexité, évitant qu'un seul batch ne concentre l'essentiel de la distance totale parcourue. Le cluster C0 (30 articles) est légèrement sur-représenté, ce qui pourrait être optimisé en ajustant  $k$  ou en ajoutant une contrainte de taille dans le clustering.

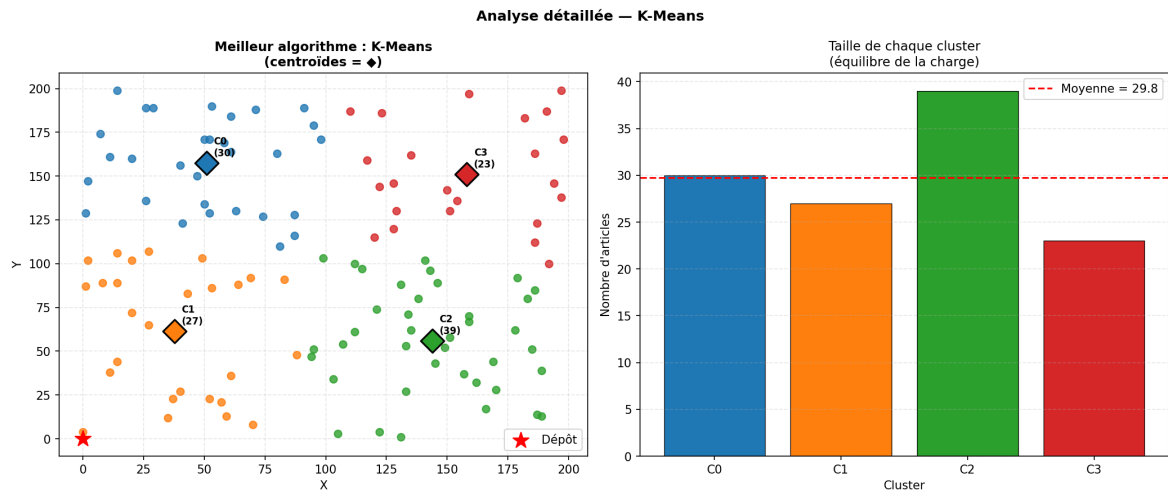


FIGURE 2.3 – K-Means ( $k=4$ ) : carte de l'entrepôt avec centroïdes et distribution des tailles de clusters.

## 2.7 Intégration avec la métaheuristique

Le module ML transmet à la métaheuristique un dictionnaire Python structuré contenant : les batches (articles par cluster), les centroïdes, la matrice de distances de Manhattan entre centroïdes, et les coordonnées du dépôt. La métaheuristique n'a donc plus à traiter 119 points d'un seul coup, mais 1 TSP de 4 nœuds + 4 sous-TSP d'environ 30 articles chacun.

**Gain de complexité :** Sans clustering :  $119!$  permutations. Avec clustering :  $4! + 4 \times 30!$  réduction drastique de l'espace de recherche.



# Chapitre 3

## Métaheuristiques et Optimisation

Dans la continuité du pré-traitement par clustering K-Means, qui a permis de diviser notre problème initial de 119 articles en 4 sous-TSP de taille plus réduite, nous appliquons des métaheuristiques afin de déterminer la tournée optimale du préparateur au sein de chaque lot.

### 3.1 Implémentation des Algorithmes

#### 3.1.1 Algorithme des Colonies de Fourmis (ACO)

L'ACO est une métaheuristique bio-inspirée particulièrement adaptée au problème du Voyageur de Commerce (TSP). Dans notre implémentation, chaque "fourmi" artificielle construit une solution probabiliste en tenant compte de deux facteurs pour se déplacer d'un point  $i$  vers un article  $j$  :

- **La visibilité** ( $\eta_{ij}$ ) : l'inverse de la distance de Manhattan entre les deux points.
- **La trace de phéromone** ( $\tau_{ij}$ ) : l'historique des meilleurs chemins trouvés.

La probabilité de transition est donnée par :

$$P_{ij} = \frac{(\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta}{\sum_k (\tau_{ik})^\alpha \cdot (\eta_{ik})^\beta}$$

Ce mécanisme permet une convergence progressive vers des solutions de qualité.

#### 3.1.2 Algorithme Génétique (AG)

L'algorithme génétique repose sur une population de solutions (chromosomes). Afin de respecter les contraintes du TSP, nous avons utilisé l'opérateur **Order Crossover (OX)**, garantissant l'absence de doublons. Une mutation par inversion est également introduite afin de maintenir la diversité et éviter les minima locaux.

### 3.2 Résultats et Comparaison des Approches

Nous avons comparé quatre scénarios :

1. **ACO sans ML**
2. **AG sans ML**
3. **ACO avec ML (K-Means, k=4)**
4. **AG avec ML (K-Means, k=4)**

### 3.2.1 Tableau de synthèse

| Approche    | Distance | Temps (s) |
|-------------|----------|-----------|
| ACO sans ML | 2588     | 3.32      |
| AG sans ML  | 12052    | 5.37      |
| ACO avec ML | 3334     | 1.34      |
| AG avec ML  | 5600     | 4.42      |

TABLE 3.1 – Comparaison des performances

### 3.2.2 Analyse des performances

L'ACO sans Machine Learning produit la meilleure distance (2588), car il optimise une tournée globale. Cependant, son temps de calcul reste élevé.

L'algorithme génétique sans Machine Learning est fortement pénalisé par l'explosion combinatoire, ce qui entraîne une distance très élevée et un temps important.

L'introduction du clustering (K-Means) permet de réduire significativement le temps de calcul. L'ACO avec ML divise le temps par plus de deux (1.34 s), au prix d'une légère augmentation de la distance (3334), due aux retours au dépôt.

L'AG avec ML améliore ses performances par rapport à sa version sans ML, mais reste moins efficace que l'ACO.

### 3.2.3 Visualisation des performances

Afin de mieux interpréter ces résultats, nous avons représenté graphiquement les performances des différentes approches.

**Comparaison globale** La figure suivante montre simultanément la distance et le temps :

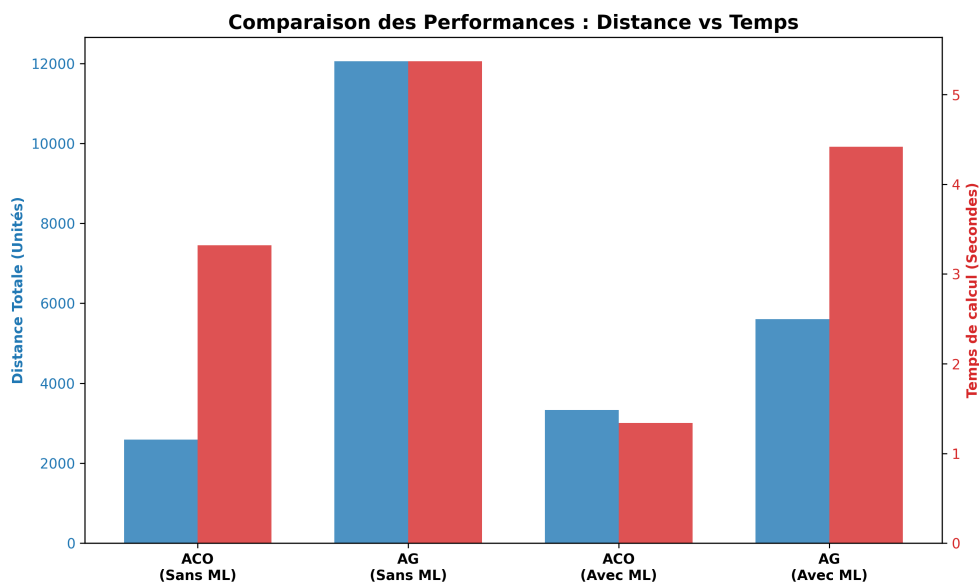


FIGURE 3.1 – Comparaison des distances et temps de calcul

On observe que :

- l'ACO sans ML minimise la distance,
- l'ACO avec ML réduit fortement le temps,
- l'AG sans ML est le moins performant,
- l'AG avec ML améliore ses résultats mais reste inférieur à l'ACO.

**Compromis Temps / Distance** La figure suivante permet d'analyser le compromis :

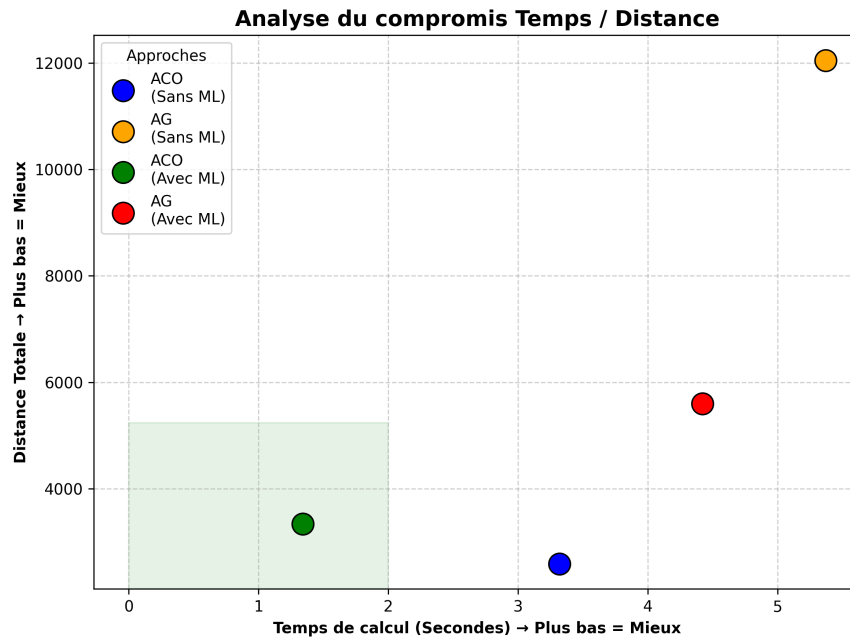


FIGURE 3.2 – Compromis temps / distance

Dans ce plan, la zone idéale se situe en bas à gauche.

On observe que :

- l'ACO avec ML est le meilleur compromis,
- l'ACO sans ML est très performant mais plus lent,
- l'AG reste globalement moins efficace.

### 3.3 Interprétation

L'augmentation de la distance dans les approches hybrides ne constitue pas une faiblesse, mais reflète une modélisation plus réaliste des contraintes logistiques (retours au dépôt).

L'ACO s'impose comme la métaheuristique la plus adaptée au problème de routage, grâce à son mécanisme d'apprentissage progressif.

À l'inverse, l'algorithme génétique nécessite un réglage plus fin pour atteindre des performances comparables.

# Chapitre 4

## Conclusion

Ce projet a démontré que l'hybridation entre le Machine Learning et les métaheuristiques constitue une approche efficace pour résoudre des problèmes d'optimisation combinatoire complexes.

L'apport principal du Machine Learning ne se limite pas à améliorer les résultats : il transforme fondamentalement la nature du problème en le rendant traitable. Face à la complexité combinatoire, la clé n'est pas toujours de chercher un algorithme plus puissant, mais de poser le problème différemment.

L'approche hybride ACO + K-Means apparaît comme la solution la plus pertinente, offrant un excellent compromis entre rapidité, qualité de solution et réalisme opérationnel. Cette architecture pourrait être étendue à des contextes plus complexes, tels que des entrepôts avec contraintes de capacité ou des commandes dynamiques en temps réel.

# Chapitre 5

## Ressources et code source

L'intégralité du code source de ce projet est disponible sur GitHub :

<https://github.com/kenza-menad/Optimisation-combinatoire>